第 3 章 解答

CH03-Q01 先看低八位，再判斷有號範圍

運算	數學上的有號結果	低八位	低八位的有號值	有號溢位
`0x7f+0x01`	128	`10000000`	−128	有
`0x80+0xff`	−129	`01111111`	127	有
`0x80-0x01`	−129	`01111111`	127	有

八位元二補數範圍是 [−128,127]，所以三列的數學結果都無法表示。第三列以 `0x80+0xfe+1` 實作，保留低八位仍為 `0x7f`。

令 c_7 進入最高位、c_8 離開最高位，則有號溢位為 $ov = c_7 \oplus c_8$。三列的進位對依序為 (1,0)、(0,1)、(0,1)，均得 1。c_8 單獨判斷的是無號加法的超界，不能代替此 XOR。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CH03-Q02

初始 $A = 0000$、$Q = 1011$、$M = 1101$。每一回合先把 加法結果 寫成 `C:A`，再把 `(C,A,Q)` 右移。

輪次	原 Q_0	加法結果 $C:A$	右移後 A	右移後 Q
1	1	`0:1101`	`0110`	`1101`
2	1	`1:0011`	`1001`	`1110`
3	0	`0:1001`	`0100`	`1111`
4	1	`1:0001`	`1000`	`1111`

第 2 回合的完整 九位元右移輸入 是 `1:0011:1101`，右移後低八位 是 `10011110`，所以 `A=1001`、`Q=1110`。C 只在該回合的加法中間存在；它隨右移進入 A，不是另外再累加第二次進位。最後 `(A,Q)=10001111₂=143=13×11`。

固定初始乘數為 $q < 2^n$，第 k 回合完成後 $R_k = 2^n A_k + Q_k$。歸納不變量為

$$R_k = \lfloor q/2^k \rfloor + M(q \bmod 2^k)2^{n-k}.$$

基底 $A_0 = 0, Q_0 = q$ 成立。$k < n$ 時第二項為偶數，所以舊 Q_k 的最低位為原 q 的第 k 位 q_k。加法與保留完整進位的右移實作

$$R_{k+1} = \lfloor (R_k + q_k M 2^n)/2 \rfloor = \lfloor q/2^{k+1} \rfloor + M(q \bmod 2^{k+1})2^{n-k-1},$$

其中使用 $q \bmod 2^{k+1} = q \bmod 2^k + q_k 2^k$。因此 $R_n = Mq$，A保存乘積高半部，Q保存低半部。舊Q0已在本回合處理並丟棄，下回合的判斷位是移入新Q0的舊Q1。數例的 $R_0, \ldots, R_4$ 為11、109、158、79、143，與每列暫存器一致。

32 位元循序實作的單筆延遲是

$$T_{\text{serial}} = 3 + 32(1 + 1) + 2 = 69 \text{ cycles}.$$

若系統藉由足夠硬體或交錯執行狀態，使獨立乘法可每四週期啟動一筆，穩態吞吐量為 $1/4 = 0.25$ 次乘法／週期，單筆延遲 仍是 69 cycles。不同交易須各自保存部分乘積與控制狀態，各階段也須避免資源衝突。只複製 ALU 並不足以建立題目假設的啟動間隔。

CH03-Q03

初始 $A = 00000$、$Q = 1101$、$M = 00101$。

輪次	合併左移後 A, Q	$A - M$	判斷後
1	`00001,1010`	-4	恢復；`00001,1010`
2	`00011,0100`	-2	恢復；`00011,0100`
3	`00110,1000`	+1	保留；`00001,1001`
4	`00011,0010`	-2	恢復；`00011,0010`

1. 各輪狀態如表。
2. 最後 $A = 00011_2 = 3$、$Q = 0010_2 = 2$。故 $13 \div 5$ 得商 2、餘數 3。
3. $5 \times 2 + 3 = 10 + 3 = 13$，且 $0 \le R < 5$，成立。
4. `-2147483648 / -1` 的 精確商 $+2147483648$ 超出 32 位元有號上界 2147483647，是數學上的溢位邊界；在本章傳統 MIPS32 不會由 `div` 自動產生 溢位例外，HI/LO 結果不可依賴，必須先用軟體檢查。其他 ISA 的例外政策需另行確認。

11÷3：獨立數例與一般證明

k	新輸入位	P_k	T	T−3	q_k	A_k	整個 Q（4 位）
0	—	0	—	—	0	0	`1011`
1	1	1	1	−2	0	1	`0110`
2	0	2	2	−1	0	2	`1100`
3	1	5	5	2	1	2	`1001`
4	1	11	5	2	3	2	`0011`

第三輪 q_k=1，整個 Q 卻是 9；最高位還含未讀輸入。第四輪後才有 Q=q_4=3，且 11=3・3+2、0≤2<3。

一般情況，令 T=2A_k+d、b=1 當且僅當 T≥M。則 $P_{k+1} = 2P_k + d = (2q_k + b)M + (T - bM)$。由 A_k<M 得 T<2M，因此新餘數 T−bM 仍落在 [0,M)。起始皆為零，歸納到 n 輪即得商餘數規格。

向零截斷：−11÷3 得 Q=−3、R=−2；−11÷(−3) 得 Q=3、R=−2。兩式分別是 −11=(−3)・3−2、−11=3・(−3)−2，且 |R|<3。向下取整的 −11÷3 則是 Q=−4、R=1：−11=(−4)・3+1。符號規則改變，不能只沿用原商再自行改餘數符號。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH03-Q04 編碼時先正規化，解碼時先分類

$12.5 = 1100.1_2 = 1.1001_2 \times 2^3$。負號使 s=1；指數欄為 3+127=130=`10000010`；fraction 為 `1001` 後補 19 個零。因此完整欄位為 `1 | 10000010 | 10010000000000000000`，每四位分組得 **`0xc1480000`**。

`0x3f000000` 的 s=0、E=126、F=0，屬正常值，所以數值為 $(1+0)2^{126-127} = 0.5$。

正數邊界	E 的八位元	F 的整數值	解碼值
最小正規正數	`00000001`	0	2^{-126}
最小次正規正數	`00000000`	1	2^{-149}

`0x7f800000` 的 E 全一、F=0，為 +∞。`0x7fc00001` 的 E 全一、F 非零，為 NaN；此格式的最高 尾數欄（fraction） 位為 1，表示 靜默 NaN（quiet NaN）。NaN 不應套用正常值公式。

CH03-Q05 中點先看距離，再看最低保留位

binary32 的 p=24，在 [1,2) 的相鄰間距為 2^{-23}。因此 $1 + 2^{-24}$ 位於 1 與 $1 + 2^{-23}$ 正中間。

候選 1 的最低保留位 L=0，另一候選 L=1。丟棄部分是半個 ULP，故 GRS=`100`；RNE 選 L=0 的 1，結果為 `0x3f800000`。精確值與結果不同，因此 inexact=1；這不是記錄誤差大小的旗標。

若只把這個和留在較寬格式、最後仍捨入成 binary32，結果依然是 1。差別要在較早捨入之前繼續運算才可能顯現：較寬格式先精確保存 $1 + 2^{-24}$，再減 1 可得 2^{-24}；先捨成 binary32 的 1 再減 1 則得 0。不能把「中間值不同」直接當成「最後同格式的這個加法結果不同」。

CH03-Q06 指數欄相加時，不能把 bias 加兩次

輸入 $a = 1.5 = 1.1_2 \times 2^0$、$b = 2^{-126} = 1.0_2 \times 2^{-126}$，均為可精確表示的正常值。

有效數的精確乘積為 $1.1_2 \times 1.0_2 = 1.1_2$，未偏移指數為 0−126=−126。乘積已在 [1,2)，所以本例不需左移，也不需右移正規化；結果 $1.5 \cdot 2^{-126}$ 仍是 正規數，編碼為 `0x00c00000`，無捨入誤差。

兩輸入的指數欄是 E_a=127、E_β=1，故本例結果欄為 $E_a+E_\beta-127$=1。直接相加會得到 128，錯把一份 bias 留在數值裡。一般乘積若超過 2，還要把正規化造成的指數加一納入；後續捨入也可能再產生進位。

CH03-Q07 同一個實數算式，不同逐步捨入

在 $x = 2^{24}$ 右側，binary32 間距為 2。x+1 位於 x 與 x+2 的中點；x 的有效數最低位為 0，RNE 選 x。所以第一式先丟掉 1，再減 x 得 0。

第二式先做 x−x=0，精確；再加 1 得 1，仍精確。實數加法的結合律沒有失效，失效的是「每步捨入後還能任意換結合順序」的推論。

對題目的延伸精度假設，若 x87 中間值精確保留 $1 + 2^{-24}$，減 1 得 2^{-24}；每步都捨為 binary32 的路徑則先得 1，再減 1 得 0。結論依賴中間精度與存回時點，不能推成所有 x87 程式都會產生同一差異。

CH03-Q08 分別計算單筆延遲與接收速率

題目模型	單筆延遲	啟動間隔	穩定接收速率
循序	3+32・1+2=37 cycles	8 cycles	1/8 operation/cycle
radix-4	6+16・1.5=30 cycles	8 cycles	1/8 operation/cycle

radix-4 的單筆結果較早完成，但題目已指定相同啟動間隔，因此穩定吞吐量相同。每八週期接受一筆依賴題目給定的複製單元或交錯 context；不能由一個未管線化的迭代單元直接推得。

相依運算必須等前一筆結果，單筆延遲尤其重要；連續獨立交易則常受啟動間隔限制。若改比較除法，還要指定商位選取、正規化、修正、每輪成本及前後處理；只有回合數減半，無法證明總延遲減半。

CH03-Q09 從丟棄位元判斷結果與例外

1. `1.0` 保留的有效數為 `1.00000000000000000000000`。加入 2^{-24} 後，丟棄位元為 `100`，即 G=1、R=0、S=0。精確值位於 1 與 $1 + 2^{-23}$ 正中；1 的最低保留位 L=0，因此 RNE 選 1，編碼 `0x3f800000`。精確值不等於結果，故設 `inexact`。
2. $3 \cdot 2^{-25} = 2^{-24} + 2^{-25}$，丟棄位元為 `110`，即 G=1、R=1、S=0。丟棄部分大於半個保留單位 2^{-23}，所以進位成 $1 + 2^{-23}$，其尾數欄最低位為 1。
3. 2^{-151} 小於最小正次正規數 2^{-149} 的一半，故捨入為 +0。2^{-150} 恰在 +0 與 2^{-149} 正中；以尾數欄整數判奇偶，0 為偶、1 為奇，所以也選 +0，編碼均為 `0x00000000`。兩者都微小且不精確，故均設 `underflow` 與 `inexact`。下溢的判定不能簡化為指數運算有無進位。
4. 最大有限值是 `0x7f7fffff`，加自身後超過有限範圍。在本題 RNE 與預設非陷阱處理下，結果是 +∞（`0x7f800000`），並設 `overflow`、`inexact`。$0 \times \infty$ 則是無效運算（invalid operation），預設回傳靜默 NaN 並設 `invalid`；「乘數有零就回傳零」不適用。

CH03-Q10 差別來自中間捨入

編碼。 $a = b = 1 + 2^{-23}$ 的指數欄為 127、尾數欄最低位為 1，所以編碼為 `0x3f800001`。$c = -(1 + 2^{-22})$ 的尾數欄為 `0x000002`，加符號位後為 `0xbf800002`。

分開乘加。 精確乘積為 $ab = (1 + 2^{-23})^2 = 1 + 2^{-22} + 2^{-46}$。此處 binary32 間距為 2^{-23}；2^{-46} 遠小於半個 ULP，所以乘法先捨入為 $1 + 2^{-22}$。再加 c 恰好相消，RNE 得 +0（`0x00000000`）。第一次捨入丟掉的項無法由後續加法恢復。

融合乘加（FMA）。 先保留精確中間結果 $(1 + 2^{-22} + 2^{-46}) - (1 + 2^{-22}) = 2^{-46}$，最後才捨入。$2^{-46}$ 可由 binary32 精確表示，是正規數；未偏移指數為 −46，指數欄為 127−46=81=`0x51`，符號位與尾數欄皆零，故編碼為 `0x28800000`。

非結合性。 令 $x = 2^{24}$。右側相鄰間距為 2，x+1 恰在 x 與 x+2 正中；RNE 選最低保留位為偶的 x。因此先算 x+1 再減 x 得 0，先算 x−x 再加 1 卻得 1。改變的是中間捨入；精確實數加法的結合律不能直接套到每步只保留 24 個有效位的運算。

CH03-Q11

1. a=$1.000_2 \times 2^0$，b=$1.001_2 \times 2^{-4} = 0.0001001_2 \times 2^0$，和為 1.0001001_2。保留 1.000，L=0、G=1、R=0，尾端 01 的 OR 為 S=1。相鄰候選 1 與 9/8 的中點是 17/16=136/128；精確和 137/128 在中點右方，所以選 9/8，絕對誤差 |144/128−137/128|=7/128。忽略 S 會把 GRS 誤當 100，選偶的 1，誤差 9/128，超過 半個 ULP=1/16。
2. 同指數相減後為 $0.001_2 \times 2^{-1}$。左移依序得 $0.010_2 \times 2^{-2}$、$0.100_2 \times 2^{-3}$、$1.000_2 \times 2^{-4}$，數值始終為 1/16。最終沒有非零丟棄位，inexact=0。
3. 1.1111_2 的候選為 1.111_2（有效數整數 15，奇）與 10.000_2（整數 16，偶）。GRS=100、L=1，選後者，再正規化為 $1.000_2 \times 2^1$。對 -1.0011_2，大小的候選為 1.001_2=9/8（整數 9，奇）與 1.010_2=5/4（整數 10，偶）；精確大小 19/16 位於中點，故選 −5/4=-1.010_2。加一作用在大小上，帶回負號後數值更負，不能稱為向 +∞ 捨入。兩題皆設 inexact。

CH03-Q12

1. $2^{-126}/2=2^{-127}=(2^{22})\delta$，尾數欄整數 $F=2^{22}$，指數欄=0，編碼為 0x00400000。結果精確，underflow=0、inexact=0。

δ/2 的候選是 0（F=0，偶）與 δ（F=1，奇），位於正中，故得 +0，編碼為 0x00000000。它微小且不精確，所以 underflow=1、inexact=1。

3δ/2 的候選是 δ（F=1，奇）與 2δ（F=2，偶），故得 2δ，編碼為 0x00000002；精確值不等於結果，同樣 underflow=1、inexact=1。

2. 最大次正規正數 為 $(2^{23}-1)\delta$，最小正規正數 為 $2^{23}\delta$。兩者相減正好 δ，編碼為 0x00000001；這次減法精確，所以兩個旗標皆為 0。輸出很小並不足以判定 underflow 旗標。

3. 指數欄=0 時，值為 $(F/2^{23})\cdot 2^{-126}=F\delta$；F 每增一，值增 δ。最後一個次正規數與第一個正規數 也只差 δ，所以沒有突然變大的缺口。次正規數的固定絕對間距不代表固定相對精度：越靠近 0，同一個 δ 占數值的比例越大。

CH03-Q13

$M=-8$ 擴位為 `11000`，起始狀態是 `00000 1101 0`。正確追蹤如下：

輪次	舊 Q_0q_{-1}	A 的操作	加減後 A Q q_{-1}	右移後 A Q q_{-1}
1	`10`	−M	`01000 1101 0`	`00100 0110 1`
2	`01`	+M	`11100 0110 1`	`11110 0011 0`
3	`10`	−M	`00110 0011 0`	`00011 0001 1`
4	`11`	0	`00011 0001 1`	`00001 1000 1`

最後丟掉歷史位，$(A,Q)=$`00001 1000`；移除冗餘符號位得 `00011000`，值為 24。乘數的重編碼為 $-1+2-4=-3$，乘積為 $8-16+32=24$。

四位元路徑為何失敗？

若只用四位元，第一輪要求 $0-(-8)=8$，但 `1000` 被解讀為 -8。錯誤追蹤是：

輪次	舊 Q_0q_{-1}	A 的操作	加減後 A Q q_{-1}	右移後 A Q q_{-1}
1	`10`	−M	`1000 1101 0`	`1100 0110 1`
2	`01`	+M	`0100 0110 1`	`0010 0011 0`
3	`10`	−M	`1010 0011 0`	`1101 0001 1`
4	`11`	0	`1101 0001 1`	`1110 1000 1`

最後 `11101000` 代表 -24，不是 24。問題在第一輪的有號溢位；之後正確地算術右移也無法恢復已遺失的符號資訊。

加減次數與週期數

`0101` 的低位到高位係數是 $(-1,+1,-1,+1)$，$-1+2-4+8=5$，所以有四次加減。無號位元中只有兩個 1，只需兩次加法。本文規定四輪，不代表四個週期：還須知道加減、移位與控制是否同週期完成。

另一組 radix-4 數例

六位元 -13 是 `110011`，補右側歷史位 0。三組依序為 $(q_1q_0q_{-1})$=`110`、$(q_3q_2q_1)$=`001`、$(q_5q_4q_3)$=`110`，係數為 $(-1,+1,-1)$，故 $-13=-1+4-16$。

j	係數	部分乘積	十二位元表示
0	−1	+5	`000000000101`
1	+1	−20	`111111101100`
2	−1	+80	`000001010000`

總和 $5-20+80=65$，十二位元為 `000001000001`。將 $d_{2j}=q_{2j-1}-q_{2j}$、$d_{2j+1}=q_{2j}-q_{2j+1}$ 代入，得到 $c_j=q_{2j-1}+q_{2j}-2q_{2j+1}$；所以八種群組的係數由同一條代數式決定。

CH03-Q14

1. 每一層都能核對總和

列	八位元	無號值
P_0	00001011	11
P_1	00010110	22
P_2	00101100	44
P_3	01011000	88
$S_1=P_0$ XOR P_1 XOR P_2	00110001	49
C_1=majority(P_0,P_1,P_2) 左移一位	00011100	28
$S_2=S_1$ XOR C_1 XOR P_3	01110101	117
C_2=majority(S_1,C_1,P_3) 左移一位	00110000	48

第一層 49+28+88=165，第二層 117+48=165，恰為 11×15。majority 是逐位運算，沒有跨欄的輸入進位。

2. 最後的進位與漏移位反例

j，由低到高	0	1	2	3	4	5	6	7
$S_2[j]$	1	0	1	0	1	1	1	0
$C_2[j]$	0	0	0	0	1	1	0	0
輸入進位 c_j	0	0	0	0	0	1	1	1
結果位	1	0	1	0	0	1	0	1
輸出進位 c_{j+1}	0	0	0	0	1	1	1	0

由高到低讀得 10100101_2=165。若兩層都漏掉左移，第一層錯給 S_1=49、C_1=14；第二層對 49,14,88 算出 S_2=103、C_2=24，最後錯得 127。錯誤不是少一個尚待處理的 carry，而是把 carry 的權重減半。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3. 一欄的恆等式可以加成整列的證明

三個位元之和只能是 0,1,2,3；逐一核對得到 $x_j+y_j+z_j=s_j+2k_j$。各乘 2^j，從 j=0 到 W−1 相加，就得到 X+Y+Z=S+C，其中 C 的第 W 位也要保留。三個四位元 15 給 $S=1111_2=15$、$C=11110_2=30$，精確總和為 $101101_2=45$，需要六位；只取低四位時 C 截成 $1110_2=14$，(15+14) mod 16=13=1101_2。任意三列的精確總和最多為 $3(2^W-1)$，W≥2 時可能需要 W+2 位；W=1 的最大和為 3，只需兩位。

4. 層數與電路延遲要分開算

九列依序變成 6、4、3、2，共四層。指定模型給上界 40+4×60+180+20=480 ps。這只是使用所給延遲上界的估算；沒有對手電路的 CPA、布線、負載與製程資料，不能據此宣稱比較結果，也不是量測值。

5. 五筆獨立交易的完整時序

交易	接受邊緣	接受時間／ps	完成邊緣	完成時間／ps
A	0	0	3	750
B	1	250	4	1000
C	2	500	5	1250
D	3	750	6	1500
E	4	1000	7	1750

第一筆延遲 750 ps，啟動間隔 250 ps，最後完成時間 1750 ps。若 B 使用 A 的乘積，邊緣 1 時尚無結果，不能照原表接受完整輸入；須依實際介面延後並滿足取樣時序。獨立交易可重疊的前提失效，不是 carry-save 算術失效。

參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study